

HOSPITAL MANAGEMENT SYSTEM

Project Documentation

Java • JDBC • MySQL • Eclipse

Patients • Doctors • Departments • Appointments • Billing • Reports

Table of Contents

1. Project Overview
2. Objectives
3. Technology Stack
4. System Architecture
5. Project Structure
6. Database Design
7. Module Documentation
8. Appointment Workflow
9. Billing Workflow
10. Reports
11. Validation and Exception Handling
12. Testing
13. Current Status
14. Future Enhancements
15. Conclusion

1. Project Overview

The Hospital Management System is a Java console application connected to MySQL using JDBC. It manages patient, doctor, department, appointment, billing, and reporting operations through a layered architecture.

The main menu contains Patient Management, Doctor Management, Department Management, Appointment Management, Billing, Reports, and Exit.

2. Objectives

- Maintain patient registration and patient records.
- Maintain doctor information and department assignments.
- Maintain hospital departments with unique department names.
- Book, search, view, cancel, and review appointments.
- Display future booked appointments in a doctor's schedule.
- Generate appointment-linked bills and calculate totals.
- Track PENDING and PAID payment states.
- Produce hospital, appointment, and billing reports.
- Use validation, exception handling, JDBC, and foreign-key relationships.

3. Technology Stack

Technology	Purpose
Java	Application and business logic
JDBC	MySQL connectivity and SQL execution
MySQL	Persistent relational database
Eclipse	Development and testing environment

4. System Architecture

The project follows a layered architecture:

- app — console menus and user interaction.
- model — entity classes such as Patient, Doctor, Department, Appointment, and Bill.
- service — service interfaces.
- serviceimpl — business/service implementations.
- dao — database operation interfaces.
- daoimpl — JDBC and SQL implementations.
- validation — domain validation.
- exception — custom exceptions.
- util — shared utilities such as DBConnection and InputUtil.

Typical flow: Menu → Service → DAO → MySQL → DAO → Service → Menu.

5. Project Structure

```
src/  
├─ app/           Main and management menus  
├─ model/        Patient, Doctor, Department, Appointment, Bill
```

— dao/	DAO interfaces
— daoimpl/	JDBC/SQL DAO implementations
— service/	Service interfaces
— serviceimpl/	Service implementations
— validation/	Validation classes
— exception/	Custom exceptions
— util/	DB connection and input utilities

6. Database Design

The project uses the hospital_db database. The main tables are patients, doctors, departments, appointments, and bills.

Table	Primary Key	Important Fields	Relationship
patients	patient_id	patient_name, age, gender, phone, address, blood_group	Referenced by appointments
departments	department_id	department_name UNIQUE	Referenced by doctors
doctors	doctor_id	doctor_name, specialization, experience, consultation_fee, department_id	References departments
appointments	appointment_id	patient_id, doctor_id, appointment_date, appointment_time, status	References patients/doctors
bills	bill_id	appointment_id, consultation_fee, medicine_charge, lab_charge, total_amount, payment_status	References appointments

Foreign keys enforce relationships. For example, an appointment cannot reference a patient that does not exist, and a bill is linked to an appointment rather than directly to a patient.

7. Module Documentation

Patient Management

Add, view, search, update, delete, and return. Patient data includes name, age, gender, phone, address, and blood group.

Doctor Management

Add, view, search, update, delete, and return. Doctor data includes specialization, experience, consultation fee, and department.

Department Management

Add, view, search, update, delete, and return. Department names are unique.

Appointment Management

Book, view all, search, patient history, doctor schedule, cancel, and return.

Billing

Generate, view, search, search by appointment, update payment status, and return.

Reports

Hospital Summary, Appointment Report, Billing Report, and return.

8. Appointment Workflow

- Patient and doctor IDs must refer to existing records.
- The user enters patient ID, doctor ID, appointment date, and appointment time.
- The appointment is stored with a status such as BOOKED.
- Patient appointment history can show the patient's appointments.
- Doctor Schedule is filtered to future BOOKED appointments.
- Past appointments remain stored for historical records rather than being deleted.
- Appointments can be cancelled through the menu.

The doctor schedule query was improved to filter with status = 'BOOKED' and `TIMESTAMP(appointment_date, appointment_time) >= NOW()`, so past appointments no longer appear as active upcoming schedule entries.

9. Billing Workflow

- A bill is generated for an existing appointment ID.
- Consultation, medicine, and lab charges are entered as monetary values.
- The total is calculated from those three charges.
- New bills use PENDING as the initial payment status.
- Bills can be searched by bill ID or appointment ID.
- Payment status can be changed to PAID.
- Bill ID and patient ID are different identifiers; bills are linked to appointments.

A verified run generated a bill for appointment 6 with ₹500 consultation, ₹1,500 medicine, and ₹450 lab charges, for a total of ₹2,450. The bill was later changed from PENDING to PAID.

10. Reports

Report	Purpose
Hospital Summary	Counts of patients, doctors, departments, appointments, and bills
Appointment Report	Total, booked, and cancelled appointment counts
Billing Report	Total bills, paid bills, pending bills, and paid revenue

Reports are implemented through ReportDAO, ReportDAOImpl, ReportService, ReportServiceImpl, and ReportMenu.

11. Validation and Exception Handling

- Invalid patient IDs are rejected during appointment booking.
- Invalid doctor IDs are rejected during appointment booking.
- Invalid appointment IDs are rejected during billing.

- Duplicate department names are rejected by the database UNIQUE constraint.
- Invalid bill conditions use InvalidBillException where implemented.
- SQLExceptions are caught in DAO implementations.
- InputUtil supports integer, string, and BigDecimal input.

12. Testing and Verified Scenarios

Scenario	Observed Result
Patient registration	A new patient was added successfully and appeared in the patient list.
Appointment booking	Appointment 6 was successfully created for patient 21 and doctor 5.
Doctor schedule	Past appointments were excluded; future BOOKED appointments remained.
Invalid schedule lookup	Doctor 6 returned no appointments when none existed.
Bill generation	Bill 2 was generated for appointment 6 with total ₹2,450.
Bill search	Bill 2 was found using appointment ID 6.
Payment update	Bill 2 was successfully changed to PAID.
Appointment report	Total, booked, and cancelled counts were displayed.

13. Current Project Status

Area	Status
Patient Management	Complete
Doctor Management	Complete
Department Management	Complete
Appointment Management	Complete
Billing	Complete
Reports	Implemented and tested
JDBC/MySQL integration	Working
Main menu integration	Complete
Future doctor schedule filtering	Implemented

14. Future Enhancements

- Allow reception staff to register a new patient directly during appointment booking.
- Allow department and doctor selection by name instead of requiring IDs.
- Automatically mark past BOOKED appointments as COMPLETED while preserving history.
- Add richer doctor, department, revenue, and patient reports.
- Add stronger validation for phone numbers, dates, age, blood group, and payment status.
- Add authentication and role-based access.
- Add a GUI or web interface.
- Add automated unit and integration tests.

15. Conclusion

The Hospital Management System demonstrates a complete core hospital workflow using Java, JDBC, and MySQL. It applies layered architecture, object-oriented programming, SQL, validation, exception handling, relational database design, appointment scheduling, billing, payment tracking, and reporting.

The implementation is suitable as an academic or portfolio project and provides a strong base for future GUI, web, security, and reporting enhancements.

Source Basis

This document is based on the project code, database definitions, and execution output supplied during the development conversation. Observed behaviors are described as verified; suggested improvements are explicitly listed as future enhancements.